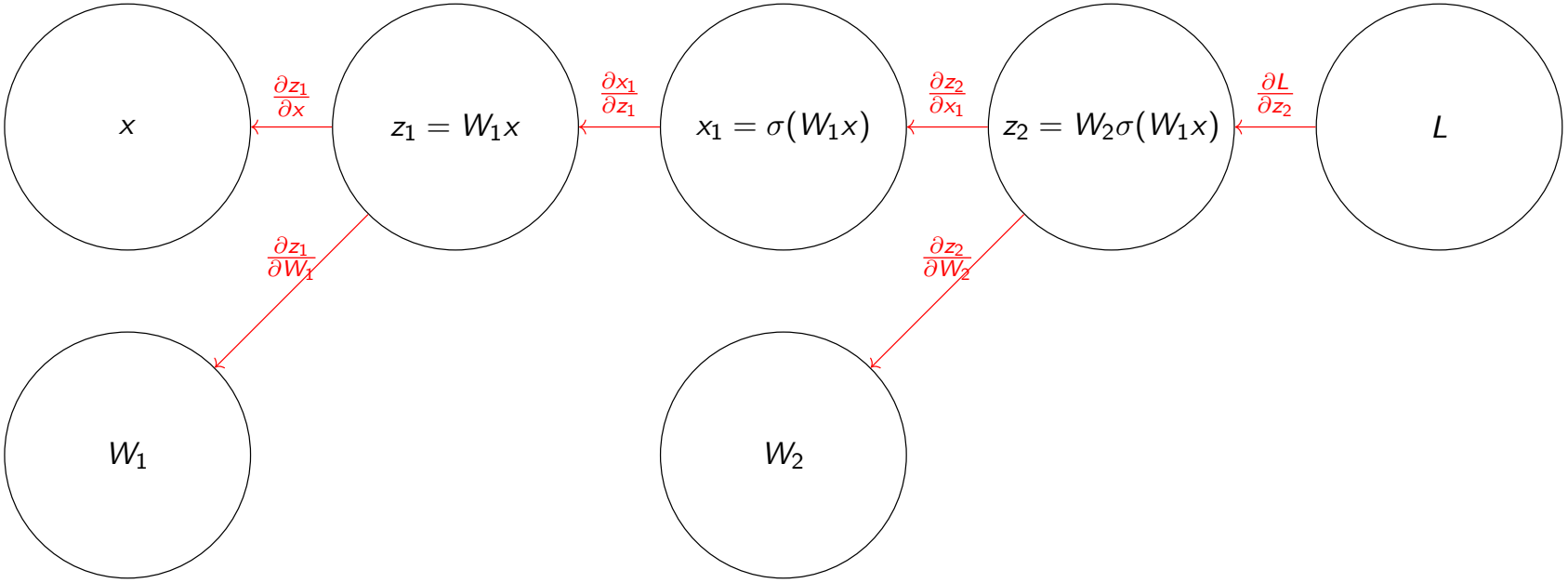
Lecture 9

- ERM: $\min_{\theta \in \Theta} L_S(\theta)$
- convex - nonconvex
- optimization methods: $\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$
- convergence: $L_S(\theta_t) \to \sim 0$ as $t \to \infty$
- How?

Introduction
Neural Networks
Optimization
Neural Networks + Optimization
Generalization
etc.

Backpropagation[25] - Computation Graph (DAG[26])



$$\underbrace{\frac{\partial L}{\partial W_1}}_{1\times d(W_1)} = \overbrace{\underbrace{\frac{\partial L}{\partial z_2}}_{1\times d(z_2)} \underbrace{\frac{\partial z_2}{\partial x_1}}_{d(z_2)\times d(x_1)}}^{\text{vector-Jacobian product}(\rightarrow)} \underbrace{\frac{\partial x_1}{\partial z_1}}_{d(x_1)\times d(z_1)} \underbrace{\frac{\partial z_1}{\partial W_1}}_{d(z_1)\times d(W_1)}$$

[25] automatic differentiation (autodiff)

[26] Directed Acyclic Graph

Backpropagation

Q. What is the time complexity of backpropagation (matmul)?
(assume $d(z_2) = d(x_1) = d(z_1) = d(W_1) = n$)

$$\underbrace{\frac{\partial L}{\partial W_1}}_{1\times d(W_1)} = \overbrace{\underbrace{\frac{\partial L}{\partial z_2}}_{1\times d(z_2)} \underbrace{\frac{\partial z_2}{\partial x_1}}_{d(z_2)\times d(x_1)}}^{\text{vector-Jacobian product}(\rightarrow)} \underbrace{\frac{\partial x_1}{\partial z_1}}_{d(x_1)\times d(z_1)} \underbrace{\frac{\partial z_1}{\partial W_1}}_{d(z_1)\times d(W_1)}$$

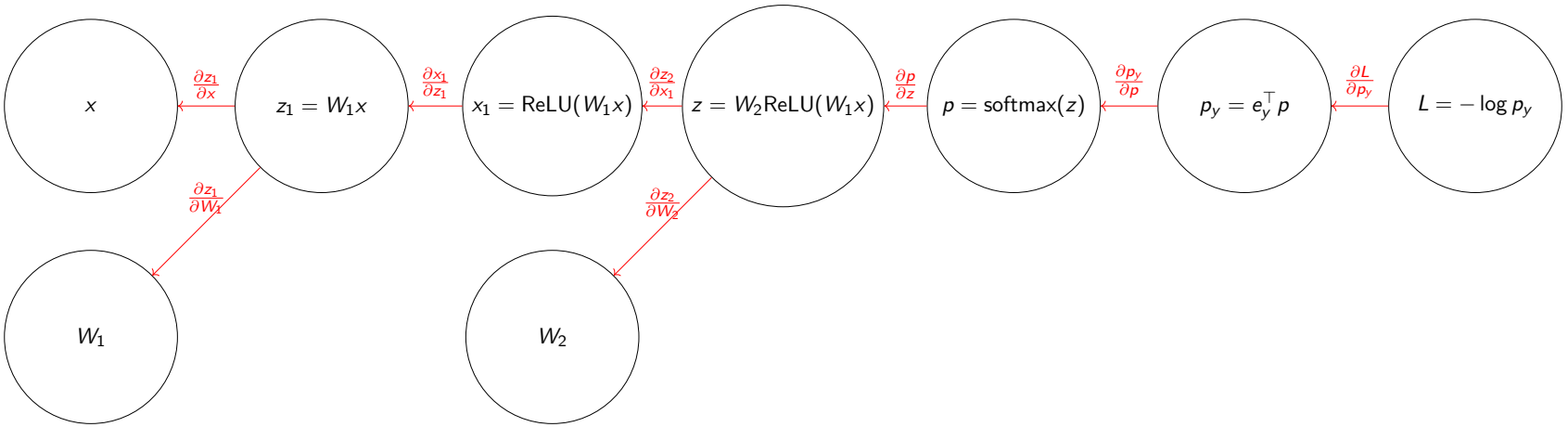
An Example - Pytorch

```
[1]  import torch
     ✓ 1.4s

[2]  x1 = torch.tensor([1.], requires_grad=True)
     x2 = torch.tensor([2.], requires_grad=True)
     x3 = torch.tensor([3.], requires_grad=True)
     ✓ 0.0s

[3]  y = x1*x2*x3
     ✓ 0.0s

[4]  y.backward()
     ✓ 1.0s

[5]  print(x1.grad)
     ✓ 0.0s

...  tensor([6.])
```

Backpropagation (Quiz)

$$(\theta, \frac{\partial L}{\partial \theta}^\top)$$

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}^\top$$



$$\frac{\partial p_y}{\partial z} = p_y(e_y - p)^\top \qquad (p = \mathrm{softmax}(z))$$

$$\frac{\partial p}{\partial z} = \mathrm{diag}(p) - pp^\top$$

$$\text{CE: } \frac{\partial L}{\partial z} = \frac{\partial[-\log(p_y)]}{\partial z} = (p - e_y)^\top \qquad (L(z, y) = -\log p_y(z))$$

$$\text{MSE: } \frac{\partial L}{\partial z} = (z - z_0)^\top \qquad (L(z, z_0) = \frac{1}{2}\|z - z_0\|^2)$$

$$\frac{\partial \mathrm{ReLU}(z)}{\partial z} = \mathrm{diag}(\mathbf{1}(z > 0))^{27}$$

$$\frac{\partial Wx}{\partial x} = W$$

[27] undefined for the elements with 0

Hyperparameter Tuning

SGD; learning rate $\eta > 0$ and mini-batch $B \sim \mathcal{P}^b$ $(|B| = b)$

$$\theta \leftarrow \theta - \eta \nabla_\theta L_B(\theta)$$

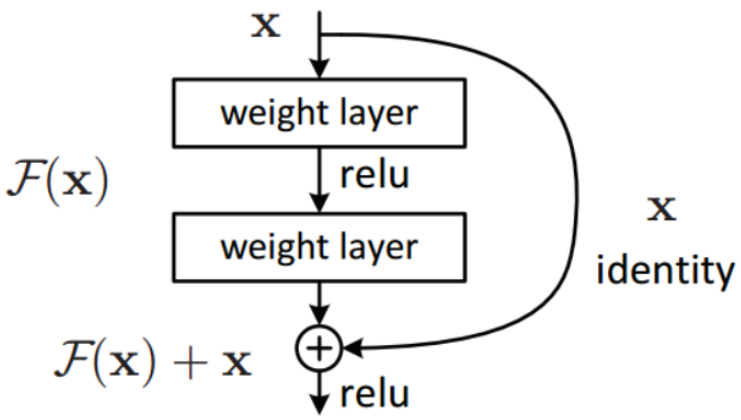
Exploding/Vanishing Gradient Problem

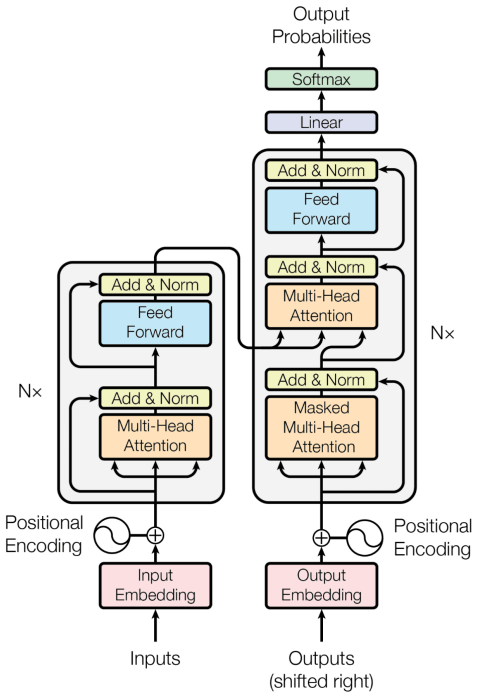
- Exploding Gradient
  - gradient clipping

$$g' = \min\left(1, \frac{c}{\|g\|}\right) g$$

    the norm of g' cannot exceed c.
- Vanishing Gradient
  - non-saturating activation functions (e.g., ReLU)
  - residual connection
  - normalization layer
  - careful parameter initialization

Residual Connection - ResNet [HZR+16]





$$y = F(x) + x$$

where $F(x)$ and x have the same shape.

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$$

Batch Normalization $(\{x^{(i)}\}_{i=1}^m \mapsto \{y^{(i)}\}_{i=1}^m)$

$$B = \{x^{(1)}, x^{(2)}, \cdots, x^{(m)}\} \qquad (x^{(i)} \in \mathbb{R}^d)$$

$$(\mu_B)_k = \frac{1}{m}\sum_{i=1}^m x_k^{(i)} \in \mathbb{R}$$

$$(\sigma_B)_k^2 = \frac{1}{m}\sum_{i=1}^m (x_k^{(i)} - (\mu_B)_k)^2 \in \mathbb{R}$$

$$\hat{x}_k^{(i)} = \frac{x_k^{(i)} - (\mu_B)_k}{\sqrt{(\sigma_B)_k^2 + \epsilon}} \in \mathbb{R} \qquad (\epsilon = \text{ 1e-5 for stability})$$

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \in \mathbb{R}^d \qquad (\text{vector notation})$$

$$y^{(i)} = \gamma \odot \hat{x}^{(i)} + \beta \qquad (\gamma, \beta : \text{ learned})$$

where $\mu_B, \sigma_B^2, \gamma, \beta$ are all vector.

Batch Normalization

During the training stage, we track the (population) running statistics using exponential moving average (EMA) with a momentum (default $\alpha = 0.1$).

$$\mu_{B_t} = \frac{1}{m} \sum_{i \in B_t} x^{(i)} \in \mathbb{R}^d$$

$$\sigma^2_{B_t} = \frac{1}{m} \sum_{i \in B_t} (x^{(i)} - \mu_{B_t})^2 \in \mathbb{R}^d \qquad \text{(vector notation)}$$

$$\text{EMA}_0 = [\mu_{B_0}, \frac{m}{m-1}\sigma^2_{B_0}] \qquad (\frac{m}{m-1} : \text{ unbiased est.})$$

$$\text{EMA}_t = \alpha[\mu_{B_t}, \frac{m}{m-1}\sigma^2_{B_t}] + (1-\alpha)\text{EMA}_{t-1}$$

pytorch/torch/nn/modules/batchnorm.py
pytorch/torch/nn/functional.py
pytorch/aten/src/ATen/native/Normalization.cpp

BN vs. LN (Layer Normalization)

- BN: across the batch axis (N) & the spatial axes (H,W)
- LN: across the channel axis (C) & the spatial axes (H,W)

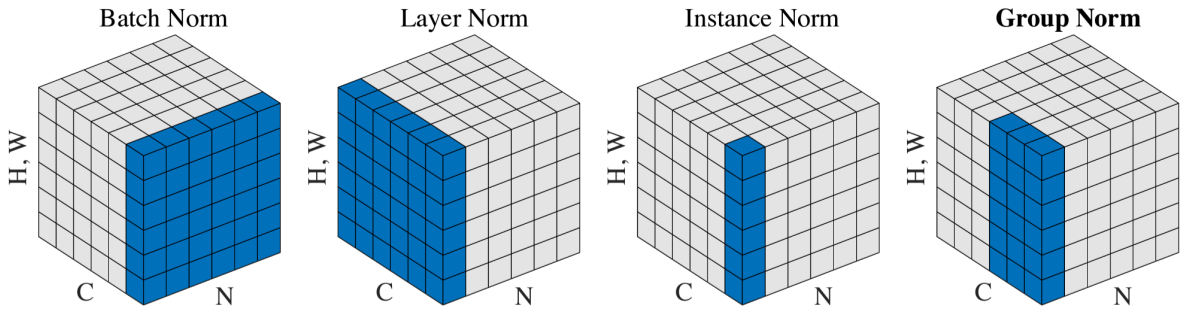


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Figure: Group Normalization

Why does BN work?

The exact reasons for the benefits of BN are still unclear. BN ...

- reduces "internal covariate shift" (?) [IS15]
  - Training DNNs is complicated because the distribution of each layer's inputs (internal covariate) changes during training, as the parameters of the previous layer changes.
- makes the optimization landscape significantly smoother [STI+18]
- reduces the sensitivity to the learning rate
- acts like a regularizer

Parameter Initialization

Sampling parameters from a standard normal with fixed variance can result in exploding activations or gradients. We want to keep the variance of the gradients from blowing up.

$$w_{ij} \sim \mathcal{N}(0, \sigma^2)$$

$$\sigma^2 = \frac{1}{n_{in}} \qquad \text{(Lecun)}$$

$$\sigma^2 = \frac{2}{n_{in} + n_{out}} \qquad \text{(Xavier/Glorot)}$$

$$\sigma^2 = \frac{2}{n_{in}} \qquad \text{(He)}$$

where n_{in}/n_{out} is the fan-in/-out of a unit (# of incoming/outgoing connections).

Lecture 10

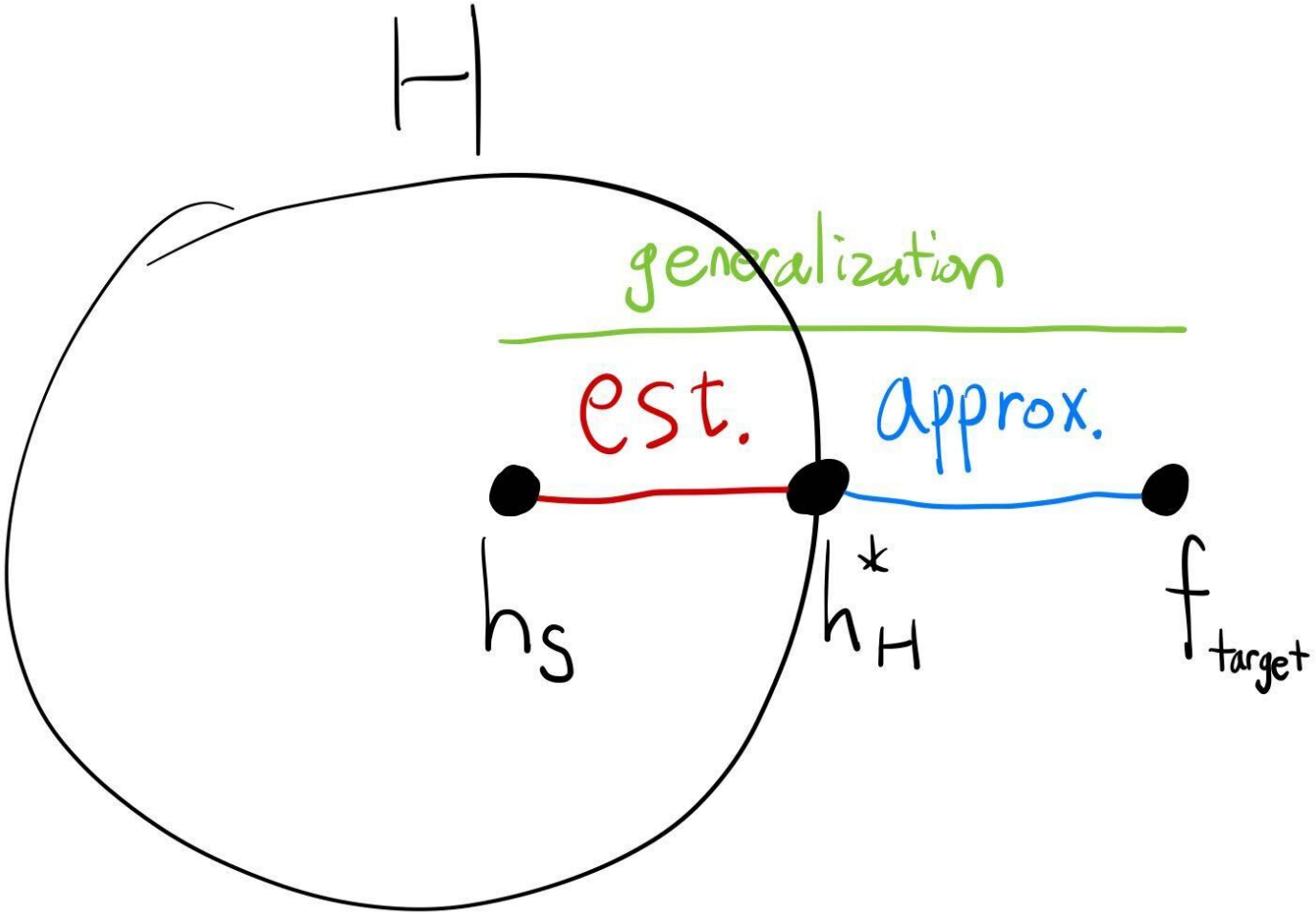
Introduction
Neural Networks
Optimization
Neural Networks + Optimization
Generalization
etc.

Generalization Error

data distribution (population) $\mathcal{P}$, sample S, hypothesis space $\mathcal{H}$

- $L_{\mathcal{P}}(h)$: generalization (true) error
- $L_S(h)$: empirical risk
- $h_S \in \arg\min_{h \in \mathcal{H}} L_S(h)$: an empirical risk minimizer over $\mathcal{H}$
- $L_{\mathcal{P}}(h_S)$: generalization (true) error of h_S
- $h^*_{\mathcal{H}} \in \arg\min_{h \in \mathcal{H}} L_{\mathcal{P}}(h)$
- $L_{\mathcal{P}}(h^*_{\mathcal{H}})$: approximation error (expressivity of $\mathcal{H}$)
  - decreases with $|\mathcal{H}|$
- $L_{\mathcal{P}}(h_S) - L_{\mathcal{P}}(h^*_{\mathcal{H}})$: estimation error
  - increases with $|\mathcal{H}|$ (?)

$$L_{\mathcal{P}}(h_S) = (L_{\mathcal{P}}(h_S) - L_{\mathcal{P}}(h^*_{\mathcal{H}})) + L_{\mathcal{P}}(h^*_{\mathcal{H}})$$



Approximation Error

data distribution (population) $\mathcal{P}$, sample S, hypothesis space $\mathcal{H}$

- $L_{\mathcal{P}}(h)$: generalization (true) error

$$\text{e.g., } L_{\mathcal{P}}(h) = \mathbb{E}_{x \sim \mathcal{P}}[(h(x) - f(x))^2]$$

- $h^*_{\mathcal{H}} \in \arg\min_{h \in \mathcal{H}} L_{\mathcal{P}}(h)$
- $L_{\mathcal{P}}(h^*_{\mathcal{H}})$: approximation error (expressivity of $\mathcal{H}$)

Theorem (Universal Approximation Theorem)

For a hypothesis space $\mathcal{H}$ of "sufficiently expressive" NNs[28], we can find $h^*_{\mathcal{H}} \in \mathcal{H}$ "close enough" to the target function f.

- *$L_{\mathcal{P}}(h^*_{\mathcal{H}})$ is small.*
- *Don't worry about the approximation error.*

[28] even for an MLP with one hidden layer

Estimation Error

$$
\begin{aligned}
L_{\mathcal{P}}(h_S) - L_{\mathcal{P}}(h^*_{\mathcal{H}}) \approx & L_{\mathcal{P}}(h_S) && \text{(test error = generalization error)} \\
& L_S(h_S) && \text{(training error)}
\end{aligned}
$$

Assumptions

- $|\mathcal{H}| < \infty$ (finite hypotheses)
- $\ell \in [0, 1]$ (bounded loss)

If we sample m i.i.d. examples ($S = \{x_i\}_{i=1}^m$) with

$$|S| \geq \left\lceil \frac{2\log(2|\mathcal{H}|/\delta)}{\epsilon^2} \right\rceil$$

and run $\text{ERM}_{\mathcal{H}}$ on this training set S, then it "learns" (small $L_{\mathcal{P}}(h_S)$).

- δ: small possibility of bad sampling
- ϵ: small $L_{\mathcal{P}}(h_S)$ (generalization error)

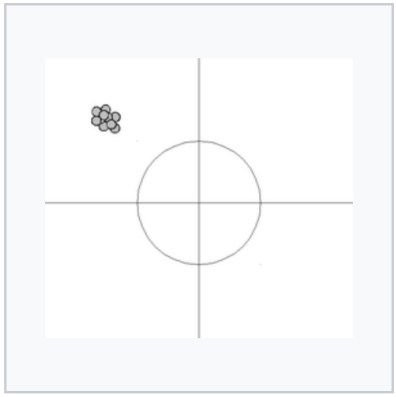
In regression, we may evaluate the quality of h_S by the expected squared difference between the true target and their predicted values, namely,

$$L_{\mathcal{P}}(h_S) = \mathbb{E}_{x \sim \mathcal{P}}[(h_S(x) - f(x))^2].$$

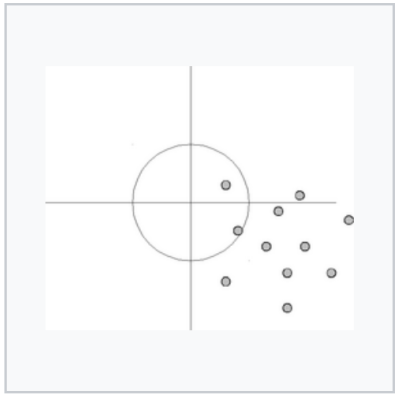
Here, we still have some randomness in sampling S.

Bias and Variance

$$\text{bias}(h_S) := \mathbb{E}_S[h_S] - f$$
$$\text{variance}(h_S) := \mathrm{Var}_S[h_S]$$

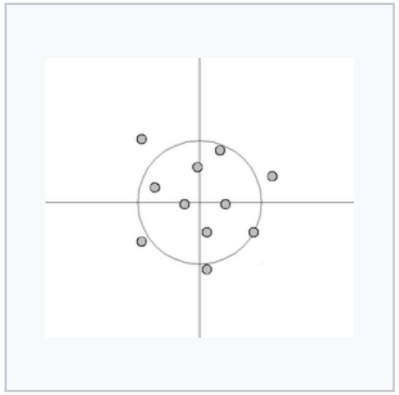


High bias, low variance



High bias, high variance

Low bias, low variance



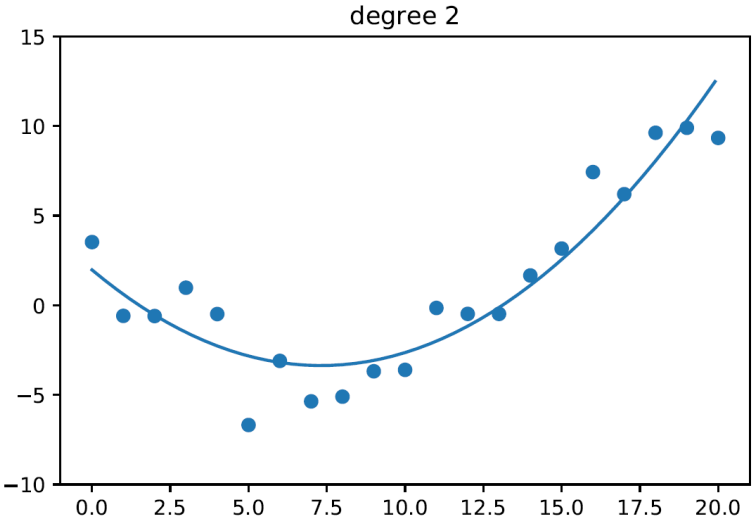
Low bias, high variance

Bias-Variance Decomposition

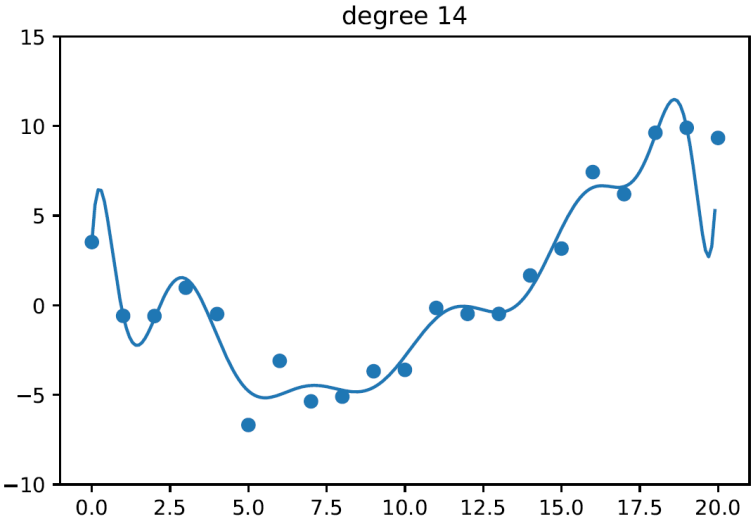
h_S: an ERM minimizer from S
f: the target function

$$
\begin{aligned}
\text{bias}(h_S) &:= \mathbb{E}_S[h_S] - f \\
&= \mu - f \\
\text{variance}(h_S) &:= \mathrm{Var}_S[h_S] \\
\mathbb{E}_S[(h_S - f)^2] &= \mathbb{E}_S[((h_S - \mu) + (\mu - f))^2] \\
&= \mathbb{E}_S[(h_S - \mu)^2 + 2(h_S - \mu)(\mu - f) + (\mu - f)^2] \\
&= \mathbb{E}_S[(h_S - \mu)^2] + 2\mathbb{E}_S[(h_S - \mu)(\mu - f)] + \mathbb{E}_S[(\mu - f)^2] \\
&= \underbrace{\mathrm{Var}_S[h_S]}_{\text{variance}(h_S)} + 2\underbrace{\mathbb{E}[h_S - \mu]}_{=0}(\mu - f) + \underbrace{(\mu - f)^2}_{\text{bias}^2(h_S)}
\end{aligned}
$$

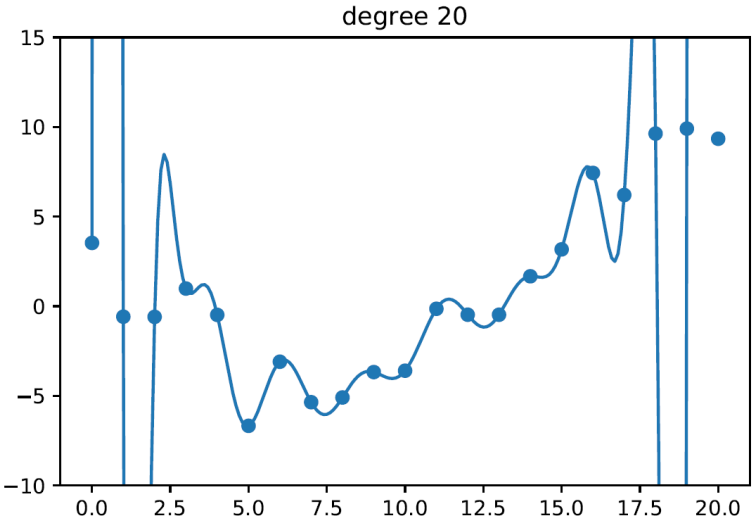
It has been widely known that the variance increases ("overfitting") and bias decreases as we increase the model complexity.


degree 2
15
10
5
0
−5
−10
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

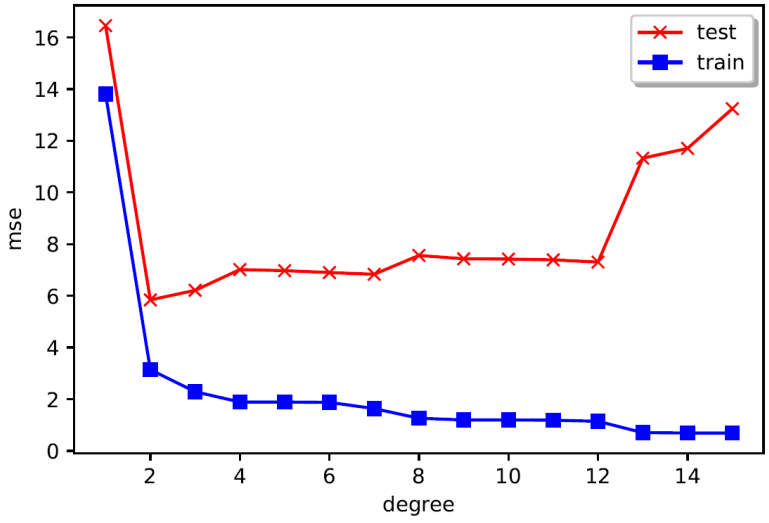
(a)


degree 14
15
10
5
0
−5
−10
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

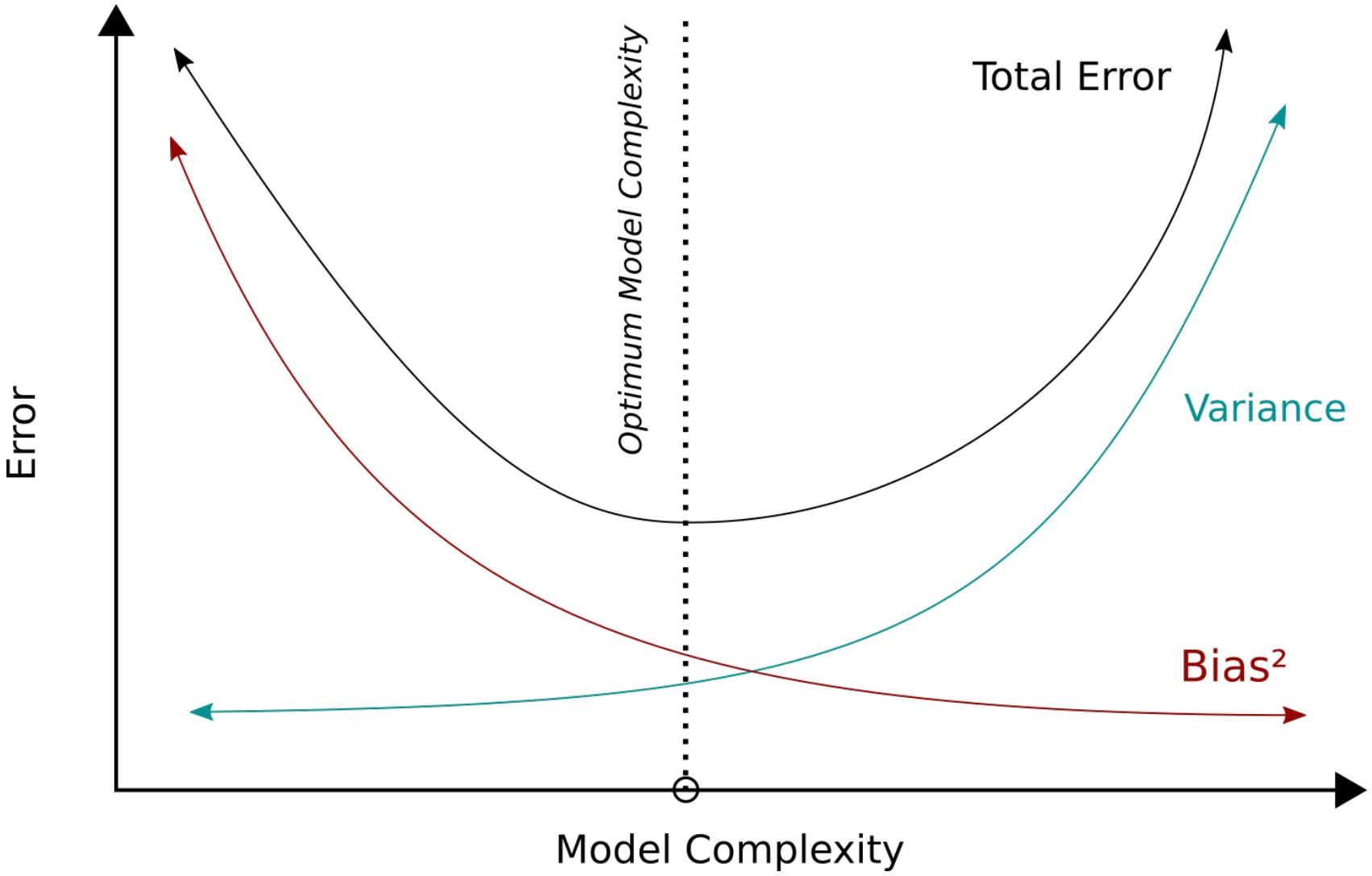
(b)


degree 20
15
10
5
0
−5
−10
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

(c)


test
train
mse
degree
16
14
12
10
8
6
4
2
0
2
4
6
8
10
12
14

(d)



bias $\sim$ training error

variance $\sim$ generalization gap $=$ total (test) error - training error

[BLL+20]
"In statistics and machine learning textbooks, an estimate that fits every training example perfectly is often presented as an illustration of overfitting ["...interpolating fits... [are] unlikely to predict future data well at all" [HTF+09]]" However, this is not true.

Regularization

- more data
- early stopping
  - when the error on the validation set starts to increase
  - cross-validation (leave-one-out)
- weight decay (MAP-Gaussian, ridge, ℓ_2)
- sparse DNNs (model compression, MAP-Laplace, LASSO, ℓ_1)
- (*) dropout: randomly (on a per-example basis) turn off all the outgoing connections from each neuron with probability p (it prevents complex co-adaptation of the hidden units; each unit must learn to perform well even if some of the other units are missing at random. This prevents the units from learning complex, but fragile, dependencies on each other.)
- (*) Bayesian neural networks (multiple possible models, yet which generalize in different ways) - Bayesian marginalization $p(y \mid x, S) = \int p(y \mid x, \theta) p(\theta \mid S) d\theta$
- implicit regularization (?)